Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3508352.3561095

INVITED-TALK

Applying GNNs to Timing Estimation at RTL

DANIELA SÁNCHEZ LOPERA, Technical University of Munich, Munich, Bayern, Germany

WOLFGANG ECKER, Technical University of Munich, Munich, Bayern, Germany

Open Access Support provided by:

Technical University of Munich

PDF Download
3508352.3561095.pdf
06 April 2026
Total Citations: 13
Total Downloads: 880

Published: 22 December 2022

Citation in BibTeX format

ICCAD '22: IEEE/ACM International Conference on Computer-Aided Design
October 30 - November 3, 2022
California, San Diego

Conference Sponsors:
SIGDA

ICCAD '22: Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design (October 2022)
https://doi.org/10.1145/3508352.3561095
ISBN: 9781450392174

Applying GNNs to Timing Estimation at RTL *(Invited Paper)*

Daniela Sánchez Lopera
Infineon Technologies AG
Technical University of Munich
Munich, Germany
daniela.sanchezlopera@infineon.com

Wolfgang Ecker
Infineon Technologies AG
Technical University of Munich
Munich, Germany
wolfgang.ecker@infineon.com

ABSTRACT

In the Electronic Design Automation (EDA) flow, signoff checks, such as timing analysis, are performed only after physical synthesis. Encountered timing violations cause re-iterations of the design flow. Hence, timing estimations at initial design stages, such as Register Transfer Level (RTL), would increase the quality of the results and lower the flow iterations. Machine learning has been used to estimate the timing behavior of chip components. However, existing solutions map EDA objects to Euclidean data without considering that EDA objects are represented naturally as graphs. Recent advances in Graph Neural Networks (GNNs) motivate the mapping from EDA objects to graphs for design metric prediction tasks at different stages. This paper maps RTL designs to directed, featured graphs with multidimensional node and edge features. These are the input to GNNs for estimating component delays and slews. An in-house hardware generation framework and open-source EDA tools for ASIC synthesis are employed for collecting training data. Experiments over unseen circuits show that GNN-based models are promising for timing estimation, even when the features come from early RTL implementations. Based on estimated delays, critical areas of the design can be detected, and proper RTL micro-architectures can be chosen without running long design iterations.

KEYWORDS

Electronic Design Automation, Timing Analysis, Register Transfer Level, Graph Neural Networks, Delay, Slew

ACM Reference Format:
Daniela Sánchez Lopera and Wolfgang Ecker. 2022. Applying GNNs to Timing Estimation at RTL *(Invited Paper)*. In *IEEE/ACM International Conference on Computer-AidedDesign (ICCAD '22), October 30-November 3, 2022, San Diego, CA, USA.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3508352.3561095

1 INTRODUCTION

The Application-Specific Integrated Circuit (ASIC) design flow is a complex and iterative process. Initial specifications are mapped to functional designs at Register Transfer Level (RTL) using Hardware Description Languages (HDLs). These are then synthesized to get a logical and physical representation using the desired CMOS technology. Both logic and physical synthesis can be seen as optimization tasks constrained by power, performance, and area results. Even though Electronic Design Automation (EDA) tools automate the design flow, they do not guarantee optimal or correct solutions. Therefore, physical verification and sign-off analysis are needed, and their findings can cause re-iterations of the flow. Thus, earlier design metric estimations can ease design-space exploration to find optimal and correct implementations. These smarter decisions at the early stages of the design result in fewer iterations of the complete EDA flow, higher Quality of Results (QoRs), and potentially, less time-to-market.

Following that motivation, Machine Learning (ML) has been incorporated into many stages of the digital design flow for solving metric prediction tasks. Notably, ML models have estimated design metrics such as congestion, wire length, hold slack, and power [16]. One of the most critical design constraints is timing. Therefore, delay estimations at the initial design steps would be beneficial. Classical methods estimating delay at early design stages are constrained to the type of components or computationally expensive for larger designs [22, 25, 27]. Shallow and deep ML methods have been applied to estimate pin-to-pin delays [33]. However, the training circuits are mapped to 1-D vectors representing pin-to-pin features related to their RTL configuration and timing behavior. This mapping loses valuable information about the circuit's connectivity. Moreover, it does not implicitly consider that delays and slews propagate across timing graphs.

Similarly, other solutions mapped EDA objects to Euclidean data, such as vectors or matrices, without considering that EDA objects are represented naturally as graphs [9, 36]. The trending Graph Neural Networks (GNNs) framework is an opportunity to solve EDA problems directly using graph structures, e.g., for intermediate RTL, gate-level, or transistor netlists. Recently, GNNs have been used in digital design stages such as placement, routing, testing, and reverse engineering. State-of-the-art results show that GNNs are superior to shallow and deep ML models, as GNNs learn from feature and topological information simultaneously [34].

Inspired by existing GNNs applications, this paper uses GNNs to predict delay and slew values of RTL components. The main contributions of this paper are:

(1) Training circuits are built using an in-house generation framework and open-source EDA tools.
(2) RTL designs are mapped to Directed Acyclic Graphs (DAGs), where the pins are the nodes and edges are their interconnections. RTL and timing-related features are represented as multidimensional vectors of nodes and edges.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ICCAD '22, October 30-November 3, 2022, San Diego, CA, USA
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9217-4/22/10.
https://doi.org/10.1145/3508352.3561095

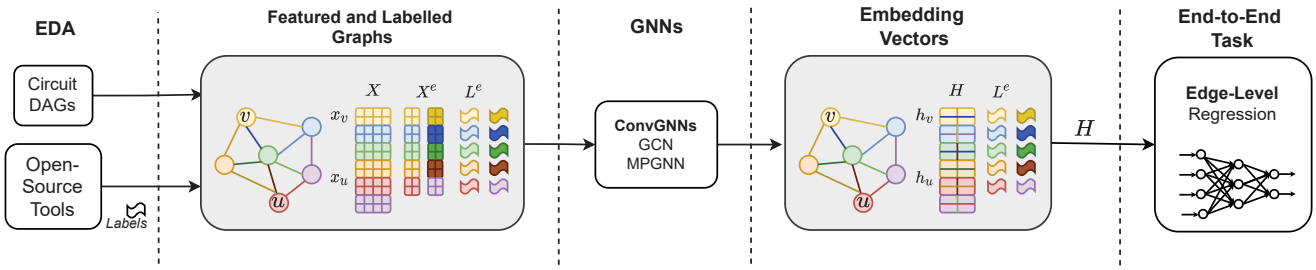


Figure 1: Delay and slew regression end-to-end GNN pipeline. Node features X, edge features X^e, and ground truth labels of edges L^e are collected using open-source EDA tools. ConvGNNs transform node and edge features and extract the final node embeddings matrix H. This is the input to an Multilayer Perceptron (MLP) trained in a supervised fashion for edge-level regression.

(3) Delay and slew prediction are formulated as an edge-level regression task solved using GNN-based models.

To that end, we follow the general end-to-end pipeline shown in Figure 1. Using estimated delays and slews, critical areas of the RTL implementation can be detected early without running the costly synthesis, layout, and timing analysis steps. Timing estimations ease design space exploration at RTL and improve the QoRs before going deeper in the design flow.

This paper is organized as follows: Section 2 briefly reviews concepts related to timing analysis, graphs, and GNNs. Section 3 surveys related works using GNNs for timing metrics prediction. The GNN solution for RTL timing prediction is presented in Section 4. The evaluation circuits are described in Section 5, and their results are discussed in Section 6. Finally, Section 7 concludes by summarizing the results and mentioning future directions.

2 BACKGROUND

In this section, we review background concepts related to Static Timing Analysis (STA), graphs, and GNNs.

2.1 Static Timing Analysis

In EDA, STA is a static method to determine whether a design works at the required clock frequency. It is called static, as no input vectors are required to simulate the complete circuit. Instead, gate and wire delays are propagated from the primary inputs to the outputs. During STA, all circuit paths are analyzed by propagating input signal transition times or slews, and considering the time needed for an input signal to go through each gate. The latter is called gate delay.

The accumulated delay at the primary output ports is the actual Arrival Time (AT). The Required Arrival Time (RAT) is fixed and calculated based on the clock period and constraints. The difference between the actual AT and RAT is called slack. This is a severity measure: a negative slack means that the signal through the path is late. The path with the worst slack is called the critical path of the circuit [15].

STA tools find critical paths of the design and whether there are timing violations. To that end, gate-level netlists and technology libraries are used. In the latter, wire and gate delay and slew models are described. Specifically, the cell delay is defined as a Lookup Table (LUT) with two variables as indexes: input *slew* and *fanout* load. The fanout is the capacitive load at the output. The slew is the time that a signal requires to make a transition and is propagated along the timing path, i.e., the transition time of the output pin of a gate is the input transition time of the following connected components in the timing graph [1].

During timing analysis, the tool uses interpolation or extrapolation to calculate delays and output slews for input values that are not specified or outside the defined index tables. Moreover, gate and wire delay models are scaled according to the current operating conditions, which physically affect the cell behavior, such as process, voltage, and temperature. These are considered fixed, and thus, STA is deterministic [2].

2.2 Graphs

Mathematically, a graph is a tuple $G = (V, E)$, where V is the set of nodes and E is the set of edges. Graph structures represent interactions or edges between related objects or nodes. An edge, denoted as $e_{u,v} \in E$, is a connection between two nodes $u, v \in V$. The neighborhood of a node v is defined as $N(v) = \{u \in V \mid (u, v) \in E\}$. A graph can be described by using an *adjacency matrix* $\mathbf{A}$, which is defined as: $\mathbf{A}_{u,v} = 1$ if $e_{u,v} \in E$, otherwise $\mathbf{A}_{u,v} = 0$. Besides the topological information provided by the $\mathbf{A}$ matrix, the nodes and edges can be complemented with feature vectors of dimensions d and c, respectively. Given a graph of n nodes and m edges, the stacked feature vectors per node build the feature matrix $\mathbf{X}$, where $\mathbf{X} \in \mathbb{R}^{n \times d}$. Similarly, the edge feature matrix is defined as $\mathbf{X}^e$, where $\mathbf{X}^e \in \mathbb{R}^{m \times c}$ [14].

Based on the direction of the edges, graphs can be classified into *undirected* and *directed* graphs. In the former, the edges have no direction and thus, the matrix $\mathbf{A}$ is symmetric. In directed graphs, the direction of edges is relevant [14]. One particular type of directed graphs are DAGs, in which the directed edges never form a closed loop [37].

Thanks to their flexibility, graphs have been used in many fields, such as chemistry [7] and computer science [30]. Classical approaches to preprocess graph-structured data use shallow embedding methods, which aim to map the position of the nodes and their neighborhood to a lower-dimensional vector [13, 20, 26]. These shallow methods are constrained. They compute an embedding representation for a given node instead of learning how to obtain

this representation in general. Moreover, they do not consider node or edge feature vectors.

2.3 Graph Neural Networks

To overcome the disadvantages of shallow methods, GNNs were introduced by Gori et al. [12] as a Neural Network (NN) framework that operates directly on graphs. Given an input graph, GNNs aim to learn the graph embedding vectors, defined as $\mathbf{h}_u \ \forall u \in V$, which encodes the neighborhood information of each node u [41]. In contrast to shallow methods, GNNs are inductive, i.e., they can generalize to unseen nodes and consider topological and feature information simultaneously. GNNs use a message-passing strategy, in which the node embedding or messages are passed through the neighbors, and a non-linear differentiable function is applied to update the node embeddings. In this way, the feature information is considered a message, and this is propagated through the local neighborhood given by the topological information.

Inspired by the advance of ML architectures such as Convolutional Neural Networks (CNNs) and mechanisms such as attention and pooling, novel GNN architectures have been proposed but keeping the message-passing strategy. In [41], GNNs are classified into four classes: Recurrent Graph Neural Networks (RecGNNs), Convolutional Graph Neural Networks (ConvGNNs), Graph Autoencoders (GAEs), and Spatial-Temporal Graph Neural Networks (STGNNs). RecGNNs are the pioneer architectures and aggregate node information recurrently by stacking computation layers and sharing learnable parameters among them. This recurrent message aggregation continues until a stable point is reached. ConvGNNs are a generalization of CNNs for graphs. In contrast to RecGNNs, layers in ConvGNNs do not share weights. GAEs and STGNNs combine different layer types, such as RecGNNs and ConvGNNs, to solve two task types: GAEs aim to reconstruct graph structures or find latent representations of graphs. STGNNs compute graph embeddings when the features vary over time.

In this paper, we use two different ConvGNNs, a Graph Convolutional Network (GCN) [39] and a Message Passing Neural Network (MPNN) [11]:

2.3.1 GCN [39]. GCNs are the most commonly used ConvGNN architecture. GCNs are considered as spectral operators, which implement the convolution of two signals in the time domain as the multiplication of these two signals in the frequency domain. Hence, spectral approaches are based on Laplacian's eigenbasis, which differs depending on the graph structure. Mathematically, the propagation along the L layers of a GCN follows Equation 1:

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right), \tag{1}$$

where $\mathbf{H}^{(l)}$ is the stacked matrix of vector embeddings h per n nodes in layer l. In $l = 0$, the embeddings of the nodes are their feature vectors, and $\mathbf{H} = \mathbf{X}$. Thus, $\mathbf{H} \in \mathbb{R}^{n \times d}$. $\sigma(\cdot)$ is a non-linear activation function. The formulation of GCNs adds self-connections to each node of the initial graph. Mathematically, this translates into adding the identity matrix of order I_n to $\mathbf{A}$, i.e., $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I_n}$. The new $\tilde{\mathbf{A}}$ is multiplied at each side by its diagonal degree matrix $\tilde{\mathbf{D}}$. The $\mathbf{W}^{(l)}$ matrix contains the learnable parameters of the l layer.

From the message-passing strategy, a GCN layer performs three steps: first, the node embeddings of a node's neighbors h_v are aggregated to build h_u. This is multiplied by the weight matrix $\mathbf{W}$ and passed through a non-linear function. These steps are repeated for each node and L layers. The main drawback of GCNs is that it does not support message passing with multidimensional edge features. In this paper, we extend GCNs, so that multidimensional edge feature vectors are considered for updating the node embeddings.

2.3.2 MPNN [11]. In contrast to GCNs, MPNNs are spatial approaches as they define convolutions directly on the graphs based on their topology. Moreover, they consider edge features $\mathbf{X^e}$. To that end, MPNN builds a *message* m_u that travels to the node u from its neighbors $v \in N(u)$ and considers the edge features $x^e_{u,v}$, as shown in Equation 2:

$$m_u^{l+1} = \sum_{v \in N(u)} M(h_u^l, h_v^l, x^e_{u,v}), \tag{2}$$

where l is the time step and M is the message function, usually implemented as an MLP. All messages arriving to the node u are aggregated using the addition operator. The message m is then used to calculate the updated node embedding h_u following Equation 3.

$$h_u^{l+1} = F(h_u^l, m_u^{l+1}), \tag{3}$$

where F is a node update function also usually implemented as a simple MLP.

3 RELATED WORK

ML Euclidean approaches have been used to estimate delays or arrival times of circuit components at the early stages of the design flow, such as at High-level Synthesis (HLS) [24], RTL [33], logic synthesis [28], and placement [6, 17, 23]. In [24], timing is modeled as a regression problem, where the input features come directly from the HLS reports. The maximum clock frequency, throughput, and area are predicted using ML models such as Random Forest (RF), support vector machines, and a NN. In [33], an MLP and a Tabular MLP (TabMLP) are used for the pin-to-pin delay and slew estimation. The training data are built using RTL features and the results of an open-source STA tool. Ramesh et al. [28] use a NN for predicting the AT of a circuit. The ML model is trained with results from a commercial STA tool. The predicted ATs are further compared to the results of a statistical STA. In [23], net delays of Field Programmable Gate Arrays (FPGAs) designs are predicted using an RF regressor. In [17], ML-based models are trained to learn wire delays and slews for incremental STA (iSTA). Finally, Barboza et al. [6] predict pre-routing wire delays and slews by exploring three ML-based regression models: Lasso, a NN, and an RF.

Even though those approaches provide good results and are fast, they build datasets mapping circuits and their features to tabular data without considering that circuits or netlists are intuitively graphs. The new advances of GNNs allow using ML techniques directly on EDA objects and exploring new architectures to improve the quality of the predictions. Thus, GNNs have already been used in many stages of the digital design flow [34]. Especially, GNNs have shown superiority for design metric prediction when estimating net length [42], routing congestion [18], and post-route Total Negative Slack (TNS) [21]. In [21], the netlists from three sequential stages

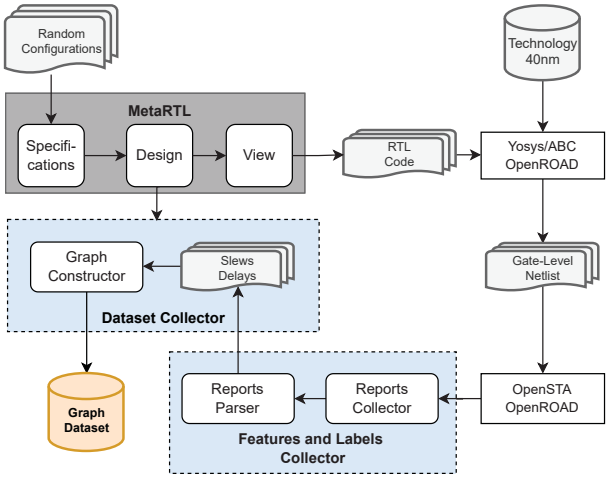


Figure 2: General Flow for Graph Dataset Generation

of the design flow are mapped to featured graphs and encoded by a global GNN. The node embeddings are then used as inputs to an MLP predicting the TNS per stage.

Inspired by [21] and [33], this paper proposes an end-to-end GNN pipeline for ASIC delay and slew prediction. Similar to [33], the input graphs and their features come directly from the earlier RTL representation. The ground-truth labels are obtained from open-source tools performing logic synthesis and STA. As in [21], we propose the timing prediction as a regression task that can be solved by GNNs. Thus, we map the slew and delay prediction as an edge-level regression task. However, the estimation of timing metrics, such as TNS at RTL remains open for future work.

4 USING GNNS FOR DELAY AND SLEW PREDICTION

Inspired by [34], we map our delay and slew regression as a supervised edge-level regression task, which general pipeline is depicted in Figure 1. This section describes our graph data generation flow and the GNN architectures used.

4.1 Dataset Generation

Figure 2 sketches the dataset generation flow. We use an in-house Hardware (HW) generator framework called *MetaRTL* [35]. MetaRTL uses meta-modeling and applies the Model Driven Architecture (MDA) principle to RTL generation [38]. In general, this framework provides three layers of a design: the initial specifications, the intermediate representation, and finally, the mapping of the design to an HDL. By providing a set of random specifications, MetaRTL generates different designs which compose the training dataset. The random configurations used were as follows: A design contains from one up to six basic components. When the number of components is higher than one, the first gate output is the input to the other gates. The number of inputs to a component varies from one to eight. Similarly, each input's bit widths vary between 1, 2, 4, and 8-bit. Finally, the component types used are inverters (NOT) and logic gates (AND, NAND, OR, NOR, XOR, and XNOR). The latter gates in a bitwise, logical, and reduced fashion.

Table 1: Node features

Feature	Size	Description
RTL Node Type	23	One-hot encoded node type
# Inputs	1	Number of input connections to the node
# Outputs	1	Number of output connections of the node
Bit width	1	Bit width of the node
Depth Level	1	Level of the node in the DAG
Edge Invertor	1	Flag indicating whether the component inverts the timing arcs

Table 2: Edge features and labels. Each one considers both edges, rising and falling, and both corners, min. and max.

Feature/ Labels	Size	Description
Input Slew	4	Input transition time from the input pin
Output Slew	4	Output transition time of the output pin
Delay	4	Delay through the gate from input to output pin

The generated Verilog code from MetaRTL is synthesized using Yosys[1] [40], an open-source tool that maps Verilog Hardware Description Language (HDL) designs to a gate-level netlist using CMOS cells of a technology library. In this case, a proprietary $40nm$ technology is used. The synthesized netlists are the input to the STA tool. The clock period was set to $6.6ns$. Moreover, clock and port configurations, such as delay and loads, were fixed during the generation of training and evaluation data. The input transition times of the primary input ports were varied to cover four different corner points. For both rising and falling edges, the four $[min, max]$ tuples considered were: $[0, 0]$, $[0.5, 0, 5]$, $[1.0, 1.0]$ and $[0.0, 1.0]$. The timing analysis is performed using OpenSTA[2], which is an open-source version of the commercial Parallax timer tool. Both Yosys and OpenSTA are built in the open-source RTL-to-GDS framework OpenROAD [4].

As shown in Figure 2, slew and delay information are collected and parsed from the reports generated by OpenSTA. The training designs are then mapped to DAGs using the pin node convention as illustrated in Figure 3. After the graph's topology is selected, the nodes and edges are annotated with numerical features listed in Table 1 and Table 2, respectively. All node and edge features and labels were normalized to be in the range of $[0, 1]$. The node feature indicating the component type was one-hot encoded, resulting in a vector of size 1×23. Notice that the number of RTL-supported components is 19. The remaining four types indicate primary port

[1]https://github.com/YosysHQ/yosys
[2]https://github.com/The-OpenROAD-Project/OpenSTA

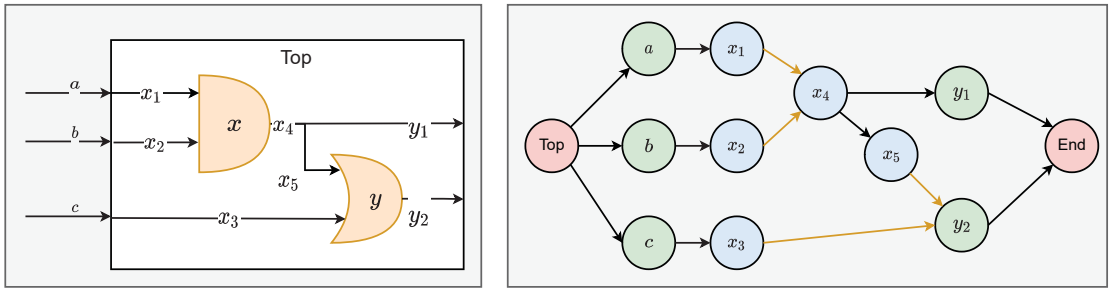


Figure 3: Sample mapping from the circuit (left side) to a DAG (right side) using pin node convention: red nodes represent starting and end points of the graph. Primary ports are colored green. Blue nodes can be either input or output pins. The type of node is one-hot-encoded as a node feature. Delays and slews of edges highlighted with yellow are the edges to predict. The remaining edges are considered ideal wires.

pins, start-end nodes, or unknown components, which might come during the evaluation of unseen designs. Hence, the node features are $1D$ vectors of 28 elements, while the edge feature vectors have a dimension of 1×4. The edge label vectors are of size 1×8.

The total number of generated designs varies depending on the component type: 803 circuits were generated for each logical and reduced gate, for each bitwise gate 774, and for the NOT gate 106. Additionally, running STA for the four input transition times produces a final dataset length of 56336 graphs. The dataset was split into training (70%), validation (20%), and test (10%) sets.

4.2 GNN Architectures

Following the pipeline in Figure 1, after collecting labeled and featured graphs, the GNN architecture is chosen. This paper employs an extended GCN and an MPNN to obtain node embeddings while considering node and edge features.

The used GCN architecture consists of three GNN layers and two dropout layers as shown on the left side of Figure 4. Each GNN layer consists of two linear layers. The first one builds a message by concatenating the embeddings of the edges and transforming them through a ReLU [3] activation function. The message is aggregated to the embedding vectors of the neighbors. The second linear layer takes this message and concatenates it to the embedding vector of the current node. The result is transformed again through a ReLU function to get a final updated node embedding. A more common GCN implementation consists of only one linear layer in each GNN module dealing with node features.

The MPNN architecture, as described in Equation 2, builds the messages using both node and edge features. The building blocks of the implemented MPNN architecture are shown on the right side of Figure 4. First, the node features are passed through a linear layer and a ReLU function for an initial node projection. Afterward, the message-passing strategy is performed two times. Each time, an edge network composed of two linear layers and a ReLU function transforms the edges features and their hidden embedding vectors. The resulting edge embeddings are the inputs to an *NNConv*, a graph convolution layer that multiplies its inputs with the node embeddings of the neighbors. This product is added to the embeddings of the current node. Finally, a Gated Recurrent Unit (GRU) [8]

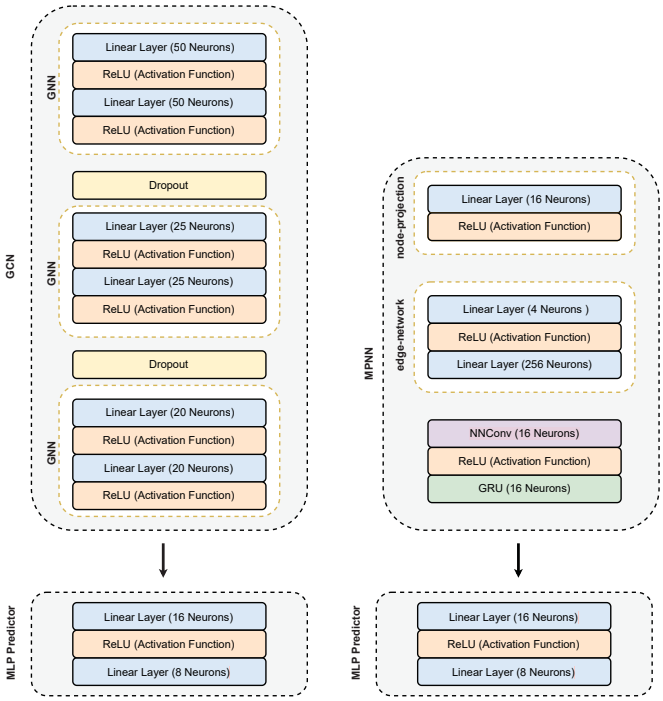
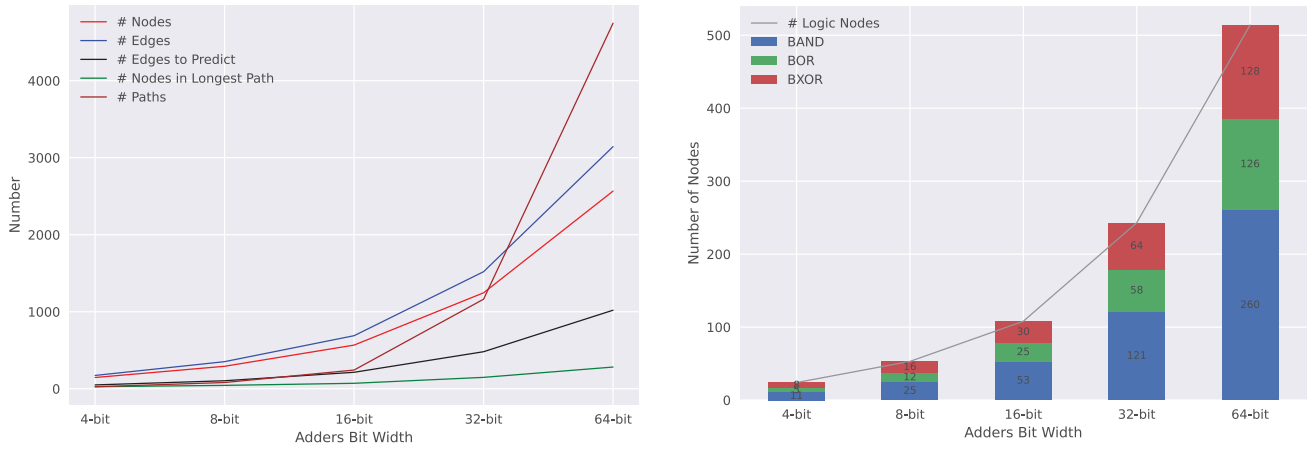


Figure 4: GNNs used: GCN-based (left side) and MPNN-based (right side)

is used to recurrently combine the current node embeddings with the node features.

The node embeddings obtained by the MPNN and the modified GCN are the inputs to an MLP predictor. Specifically, its input is a concatenated vector of the embedding vectors of the source nodes h_{src} and destination nodes h_{dst} of the edges. Thus, the input dimension of the MLP is $1 \times 2D$, where D is the size of the output node embedding vector. The number of neurons in the output layer is eight, so the MLP predicts the eight labels listed in Table 2, i.e., the maximum and minimum output slews and delays for rising and falling edges.



(a) General Number of Nodes and Edges

(b) Number of Logic Nodes

Figure 5: Averaged graph information over all adders for each bit width.

The end-to-end frameworks using a GNN and an MLP predictor are trained in a supervised setup with a stochastic gradient descent optimizer. The learning rate was $9e-3$ with a momentum of 0.9. Based on the rationale presented in [6], the loss function for slew and delay prediction should penalize under-estimations, i.e., the cases when the predicted $\hat{y}$ value is lower than the truth y value.

The common loss function used for regression is the Mean Squared Error (MSE) in Equation 4:

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2, \tag{4}$$

where N is the total number of samples. In this paper, we use the asymmetric MSE loss function introduced in [6] and formulated as in Equation 5:

$$Asymm.\ MSE = MSE + \frac{1}{N}\sum_{i=1}^{N}(|y_i - \hat{y}_i| + y_i - \hat{y}_i)^2. \tag{5}$$

To evaluate the model's performance during training, validation, and testing, we employ the MSE in Equation 4, the Mean Absolute Error (MAE) in Equation 6, and the coefficient of determination R^2 in Equation 7:

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|. \tag{6}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y}_i)^2}, \tag{7}$$

where $\bar{y}$ is the mean value. In general, the lower the MSE and MAE values, the higher the accuracy of the regression model is. R^2 is a metric in the range of $[0, 1]$ that measures how well the model fits the data: the closer to one, the better.

Both end-to-end models were implemented using Python3 and the Deep Graph Library (DGL)[3] [31] framework v0.9 with backend Pytorch™ v1.10.2 and CUDA® Toolkit version 11.1. The implementation of the MPNN model is provided in [19] and adapted to output the node embeddings of the graph and perform an edge-level regression task. Both models are trained on an NVIDIA® Tesla® P4 GPU for 100 epochs.

[3]https://www.dgl.ai/

5 CASE OF STUDY: HW ADDERS

Adders are a key component in ASIC designs and therefore also the subject of analysis and optimization through ML [32]. The Ripple Carry Adder (RCA) is the simplest way to design an adder structure for multi-bit addition. It consists of full-adder blocks connected in a chain so that the carry-out bit of the previous stage becomes the carry-in of the next block. Even though its implementation is simple and occupies a low area, its induced delay is very high. As only one full-adder block is used at a time [10].

To overcome this, more complex architectural adders perform parallel addition with prefix computation. They are called *Parallel Prefix* adders and they split the computation into three stages: Pre-processing, Carry-propagation, and Post-processing. In the first stage, *propagate* and *generate* bits are calculated for primary inputs. Then, the *carry* signals are computed in parallel to reduce delay. Finally, a simple adder obtains the total sum and output carry bit [29].

Using MetaRTL, the RCA and three prefix adders are generated: Sklansky Adder (SKA), Brent-Kung Adder (BKA), and Kogge-Stone Adder (KSA). For each type of adder, different bit widths (4, 8, 16, 32, and 64-bit) and vectorized micro-architectures are implemented. For evaluation, each design is mapped to a featured DAG.

Details of the adder DAGs are averaged over all architectures for each bit width. Figure 5(a) shows how the number of nodes, edges, edges to predict, and nodes in the longest path increase with the bit width. Similarly, Figure 5(b) shows the total number of nodes in which the node type feature is a logic gate, such as BAND, BOR, and BXOR. Nodes representing primary ports, pins, or other components are not shown. Moreover, for all DAGs, the

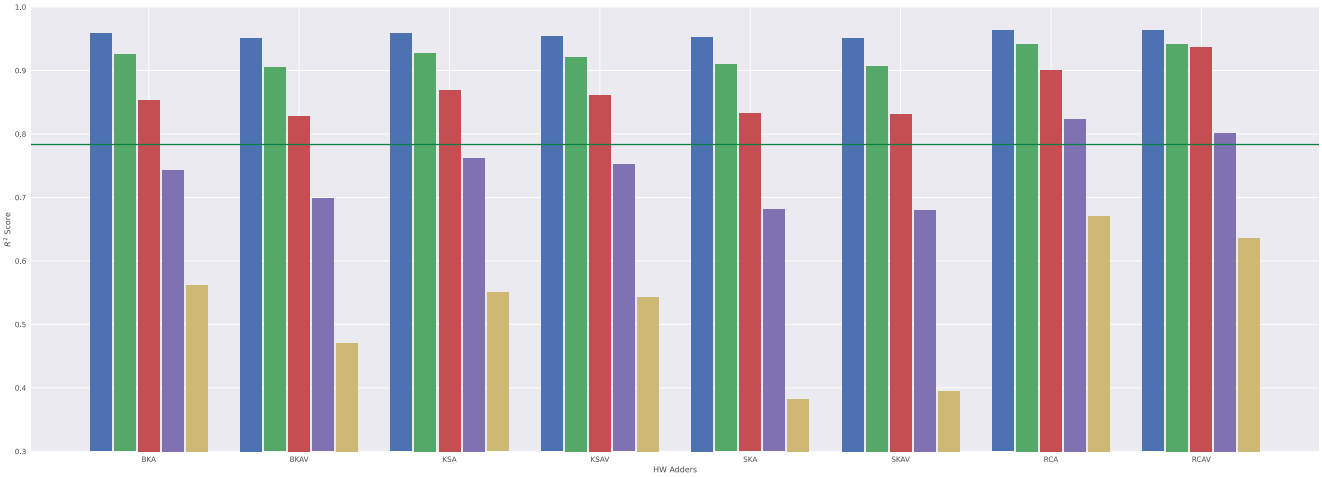


Figure 6: Average R^2 score using the GCN-based model for the delay and slew prediction over the adders and their vectorized versions (V) for all bit widths.

Table 3: Comparison with state-of-the-art results for 4-bit prefix adders

Model	R^2 [%]	MAE [%]	MSE [%]
MLP [33]	87.00	6.76	0.96
TabMLP [33]	79.00	9.49	1.39
MPNN-based	< 0	20.67	7.04
GCN-based	**95.64**	**2.64**	**0.16**

node degree with the highest frequency was two, i.e., most nodes have, on average, two incident edges. Finally, the maximum depth level for all adder graphs was four aside from the RCA and RCAV implementations, which contain up to six levels of depth.

6 RESULTS AND DISCUSSION

Both end-to-end GNN models, the MPNN and the GCN-based are used to predict slews and delays between input pins and gate nodes, i.e., values for edges connecting primary ports or input pins are not considered. The models are run for the eight implemented adders in all bit width sizes available.

The R^2 coefficient obtained by the GCN-based architecture is shown in Figure 6. For all adders and bit widths, the R^2 was, on average, 78.34%. Over all adder types, the best results are achieved for bit widths of 4-bit and 8-bit. Adders of higher bit widths contain pins with a bit width greater than 8-bit, such as 16, 32, and 64-bit, and those cases are not yet covered in the training data. The R^2 scores when using the MPNN are worse than in Figure 6.

On average, the MPNN has an MSE of 7.81% and an MAE of 22.89%. This shows that even though the MPNN considers multidimensional edge features for the node embedding updates, it performs poorly over unseen or larger graphs. This confirms the known limitation of MPNNs: they cannot distinguish two isomorphic graphs, i.e., topologically identical graphs will be embedded to the same point [5]. The GCN-based model outperforms MPNN with an averaged MSE and MAE of 0.16% and 2.76%, respectively.

Finally, we compare our results with [33]. In [33], the performance of an MLP and a TabMLP architecture for the delay and slew prediction task are evaluated. To that end, they use the 4-bit SKA, KSA, and BKA prefix adders as use cases. For the MLP model, they reported, on average, an R^2 score of 87%, MSE of 0.96%, and MAE of 6.76%. For the TabMLP using an embedding layer as the input layer, the R^2 score is, on average, 79%, the MSE is 1.39%, and the MAE is 9.49%. Table 3 shows that the GCN-based model outperforms MPNN but also the MLP-based architectures.

7 CONCLUSION

This paper applies GNNs for predicting delay and slew values of RTL components. Designs are mapped to DAGs with multidimensional node and edge features. Based on the graphs' topology, two GNNs architectures are used: MPNN and GCN. Their extracted node embeddings are passed through an MLP, which performs an edge-level regression task. Experiments over unseen circuits show that GCN-based architecture outperforms MPNN and Euclidean approaches from the state-of-the-art. The GCN-based model achieves, on average, an R^2 score of 78.34% over all designs. Results show that the R^2 score is higher than 90% for adders with a bit width less than 8-bit, which matches our expectation as higher bit widths, such as 16, 32, and 64-bit, were not considered during training.

In future work, we expect to use the estimated delays and slews to find critical and non-critical parts of an RTL design by performing a block-based STA. Similarly, we plan to find critical paths of a design directly using a GNN-based model, which instead of predicting edge-level delays provides estimations at the graph level. Moreover, we envision the generalization of our solution to cover more bit widths, component types, and complex evaluation circuits. Finally, we expect the development of more GNN architectures, which would bring more applications to the EDA field, especially targeting the prediction of design metrics at the early stages of the design flow.

ACKNOWLEDGMENTS

The work described herein is partly funded by the German Federal Ministry for Economic Affairs and Energy (BMWi) as part of the research project ProgressivKI (19A21006C).

REFERENCES

[1] 2000. *IEEE Standard for Integrated Circuit (IC) Delay and Power Calculation System.* https://doi.org/10.1109/IEEESTD.2000.7395044

[2] 2010. *Synopsys Timing Constraints and Optimization User Guide.*

[3] Abien Fred Agarap. 2018. Deep Learning using Rectified Linear Units (ReLU). https://doi.org/10.48550/ARXIV.1803.08375

[4] Tutu Ajayi and David Blaauw. 2019. OpenROAD: Toward a Self-driving, Open-source Digital Layout Implementation Tool Chain. In *Proceedings of Government Microcircuit Applications and Critical Technology Conference*. National Science Foundation, 1105–1110.

[5] Muhammet Balcilar, Pierre Héroux, Benoit Gauzere, Pascal Vasseur, Sébastien Adam, and Paul Honeine. 2021. Breaking the Limits of Message Passing Graph Neural Networks. In *International Conference on Machine Learning*. PMLR, 599–608.

[6] Erick Carvajal Barboza, Nishchal Shukla, Yiran Chen, and Jiang Hu. 2019. Machine Learning-based Pre-Routing Timing Prediction with Reduced Pessimism. In *2019 56th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.

[7] Kimberly Jordan Burch. 2019. Chemical Applications of Graph theory. In *Mathematical Physics in Theoretical Chemistry*. Elsevier, 261–294. https://doi.org/10.1016/B978-0-12-813651-5.00008-5

[8] Kyunghyun Cho, Bart Van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. 2014. On the Properties of Neural Machine Translation: Encoder-Decoder Approaches. *arXiv preprint arXiv:1409.1259* (2014).

[9] Arash Fayyazi, Soheil Shababi, Pierluigi Nuzzo, Shahin Nazarian, and Massoud Pedram. 2019. Deep Learning-Based Circuit Recognition Using Sparse Mapping and Level-Dependent Decaying Sum Circuit Representations. In *2019 Design, Automation Test in Europe Conference Exhibition (DATE)*. 638–641. https://doi.org/10.23919/DATE.2019.8715251

[10] Adnan Ghaderi and Javad Frounchi. 2016. Power and Speed Analysis of CMOS-based Multipliers using VEDIC Techniques. *IOSR Journal of Electrical and Electronics Engineering (IOSR-JEEE)* 11, 62–68.

[11] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural Message Passing for Quantum Chemistry. In *International Conference on Machine Learning*. PMLR, 1263–1272.

[12] Marco Gori, Gabriele Monfardini, and Franco Scarselli. 2005. A New Model for Learning in Graph Domains. In *Proceedings. 2005 IEEE International Joint Conference on Neural Networks, 2005.*, Vol. 2. 729–734. https://doi.org/10.1109/IJCNN.2005.1555942

[13] Aditya Grover and Jure Leskovec. 2016. Node2vec: Scalable Feature Learning for Networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, 855–864. https://doi.org/10.1145/2939672.2939754

[14] William L. Hamilton. 2020. *Graph Representation Learning*. Number 3. Morgan & Claypool Publishers. 1–159 pages.

[15] Robert B. Hitchcock, Gordon L. Smith, and David D. Cheng. 1982. Timing Analysis of Computer Hardware. *IBM Journal of Research and Development* 26, 1, 100–105. https://doi.org/10.1147/rd.261.0100

[16] Guyue Huang, Jingbo Hu, Yifan He, Jialong Liu, Mingyuan Ma, Zhaoyang Shen, Juejian Wu, Yuanfan Xu, Hengrui Zhang, Kai Zhong, Xuefei Ning, Yuzhe Ma, Haoyu Yang, Bei Yu, Huazhong Yang, and Yu Wang. 2021. Machine Learning for Electronic Design Automation: A Survey. *ACM Transactions Design Automation Electronic Systems (TODAES)* 26, 5, Article 40 (2021), 46 pages. https://doi.org/10.1145/3451179

[17] Andrew B. Kahng, Seokhyeong Kang, Hyein Lee, Siddhartha Nath, and Jyoti Wadhwani. 2013. Learning-based Approximation of Interconnect Delay and Slew in Signoff Timing Tools. In *2013 ACM/IEEE International Workshop on System Level Interconnect Prediction (SLIP)*. 1–8. https://doi.org/10.1109/SLIP.2013.6681682

[18] Robert Kirby, Saad Godil, Rajarshi Roy, and Bryan Catanzaro. 2019. CongestionNet: Routing Congestion Prediction Using Deep Graph Neural Networks. In *2019 IFIP/IEEE 27th International Conference on Very Large Scale Integration (VLSI-SoC)*. 217–222. https://doi.org/10.1109/VLSI-SoC.2019.8920342

[19] Mufei Li, Jinjing Zhou, Jiajing Hu, Wenxuan Fan, Yangkang Zhang, Yaxin Gu, and George Karypis. 2021. DGL-LifeSci: An Open-Source Toolkit for Deep Learning on Graphs in Life Science. *ACS Omega* (2021).

[20] László Lovász. 1993. Random Walks on Graphs: A Survey. *Bolyai Society Mathematical Studies. Combinatorics, Paul Erdős is Eighty* 2, 1-46 (1993), 4.

[21] Yi-Chen Lu, Siddhartha Nath, Vishal Khandelwal, and Sung Kyu Lim. 2021. Doomed Run Prediction in Physical Design by Exploiting Sequential Flow and Graph Learning. In *2021 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 1–9. https://doi.org/10.1109/ICCAD51958.2021.9643435

[22] A. Macii, E. Macii, G. Odasso, M. Poncino, and R. Scarsi. 1999. Regression-Based Macromodeling for Delay Estimation of Behavioral Components. In *Proceedings of the Ninth Great Lakes Symposium on VLSI (GLS '99)*. IEEE Computer Society, 188–191.

[23] T Martin, G Grewal, and S Areibi. 2021. A Machine Learning Approach to Predict Timing Delays During FPGA Placement. In *2021 IEEE International Parallel and Distributed Processing Symposium Workshops*. IEEE, 124–127. https://doi.org/10.1109/IPDPSW52791.2021.00026

[24] Hosein Mohammadi Makrani, Farnoud Farahmand, Hossein Sayadi, Sara Bondi, Sai Manoj Pudukotai Dinakarrao, Houman Homayoun, and Setareh Rafatirad. 2019. Pyramid: Machine Learning Framework to Estimate the Optimal Timing and Resource Usage of a High-Level Synthesis Design. In *29th International Conference on Field Programmable Logic and Applications (FPL)*. 397–403. https://doi.org/10.1109/FPL.2019.00069

[25] Mahadevamurty Nemani and Farid N Najm. 1998. Delay Estimation VLSI Circuits from a High-Level View. In *Proceedings of the 35th Annual Design Automation Conference (DAC)*. 591–594.

[26] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. DeepWalk: Online Learning of Social Representations. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, 701–710. https://doi.org/10.1145/2623330.2623732

[27] Jinhai Qiu, Sherief Reda, and Soha Hassoun. 2010. Fast, Accurate A Priori Routing Delay Estimation. In *Proceedings of the 12th ACM/IEEE International Workshop on System Level Interconnect Prediction (SLIP)* (Anaheim, California, USA). ACM, 77–82. https://doi.org/10.1145/1811100.1811119

[28] SR Ramesh and R Jayaparvathy. 2019. Artificial Neural Network Model for Arrival Time Computation in Gate Level Circuits. *Automatika: časopis za automatiku, mjerenje, elektroniku, računarstvo i komunikacije* 60, 3 (2019), 360–367.

[29] CH Sudha Rani and CH Ramesh. 2014. Design and Implementation of High Performance Parallel Prefix Adders. *International Journal of Innovative Research in Computer and Communication Engineering*, 5899–5903.

[30] Ferozuddin Riaz and Khidir M. Ali. 2011. Applications of Graph Theory in Computer Science. In *2011 Third International Conference on Computational Intelligence, Communication Systems and Networks*. 142–145. https://doi.org/10.1109/CICSyN.2011.40

[31] Yu Rong, Tingyang Xu, Junzhou Huang, Wenbing Huang, Hong Cheng, Yao Ma, Yiqi Wang, Tyler Derr, Lingfei Wu, and Tengfei Ma. 2020. Deep Graph Learning: Foundations, Advances and Applications. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, 3555–3556. https://doi.org/10.1145/3394486.3406474

[32] Rajarshi Roy, Jonathan Raiman, Neel Kant, Ilyas Elkin, Robert Kirby, Michael Siu, Stuart Oberman, Saad Godil, and Bryan Catanzaro. 2021. PrefixRL: Optimization of Parallel Prefix Circuits Using Deep Reinforcement Learning. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 853–858.

[33] Daniela Sánchez Lopera, Lorenzo Servadei, Vishwa Priyanka Kasi, Sebastian Prebeck, and Wolfgang Ecker. 2021. RTL Delay Prediction Using Neural Networks. In *2021 IEEE Nordic Circuits and Systems Conference (NorCAS)*. IEEE, 1–7.

[34] Daniela Sánchez Lopera, Lorenzo Servadei, Gamze Naz Kiprit, Robert Wille, and Wolfgang Ecker. 2022. A Comprehensive Survey on Electronic Design Automation and Graph Neural Networks: Theory and Applications. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* (2022).

[35] Johannes Schreiner, Rainer Findenigy, and Wolfgang Ecker. 2016. Design Centric Modeling of Digital Hardware. In *2016 IEEE International High-Level Design Validation and Test Workshop (HLDVT)*. 46–52. https://doi.org/10.1109/HLDVT.2016.7748254

[36] Lorenzo Servadei, Edoardo Mosca, Elena Zennaro, Keerthikumara Devarajegowda, Michael Werner, Wolfgang Ecker, and Robert Wille. 2020. Accurate Cost Estimation of Memory Systems Utilizing Machine Learning and Solutions from Computer Vision for Design Automation. *IEEE Trans. Comput.* 69, 6 (2020), 856–867. https://doi.org/10.1109/TC.2020.2968888

[37] Krishnaiyan Thulasiraman and M. N. S. Swamy. 1992. *Graphs: Theory and Algorithms*. Vol. 118. John Wiley and Sons.

[38] Frank Truyen. 2006. The Fast Guide to Model Driven Architecture.

[39] Max Welling and Thomas N Kipf. 2016. Semi-supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR 2017)*.

[40] Clifford Wolf. 2016. *Yosys Open Synthesis Suite.*

[41] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. 2021. A Comprehensive Survey on Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems* 32, 1 (2021), 4–24. https://doi.org/10.1109/TNNLS.2020.2978386

[42] Zhiyao Xie, Rongjian Liang, Xiaoqing Xu, Jiang Hu, Yixiao Duan, and Yiran Chen. 2021. Net2: A Graph Attention Network Method Customized for Pre-Placement Net Length Estimation. In *Proceedings of the 26th Asia and South Pacific Design Automation Conference (ASP-DAC)* (Tokyo, Japan). ACM, 671–677. https://doi.org/10.1145/3394885.3431562